

Softverski algoritmi u sistemima automatskog upravljanja

Support Vector Machine, stabla odlučivanja i ansambl metode

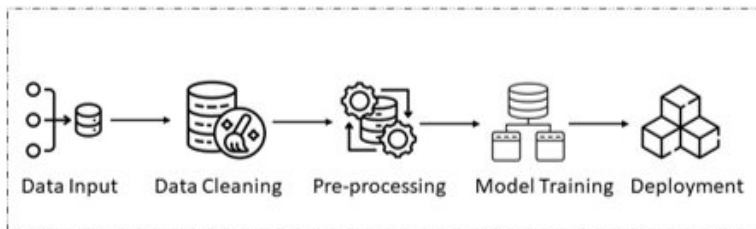
Emina Demirović

Katedra za automatsko upravljanje
Departman za računarstvo i automatiku
Fakultet tehničkih nauka

Sadržaj termina

- ▶ linearni i nelinearni modeli
- ▶ SVM - algoritam
- ▶ potporni vektori, margina i kernel
- ▶ stabla odlučivanja - algoritam
- ▶ interpretabilnost i overfitting kod stabala
- ▶ ansambl metode - ideje i pristupi
- ▶ Random Forest
- ▶ Gradient Boosting
- ▶ optimizacija hiperparametara
- ▶ Grid Search i Random Search
- ▶ validacija i poređenje modela

Fokus termina



- ▶ kako odabrati model?
- ▶ kako izvršiti fer poređenje rezultata?

Dešava se da linearni modeli nisu dovoljni za opisivanje složenijih relacija.

Zašto? Šta tada?

SVM

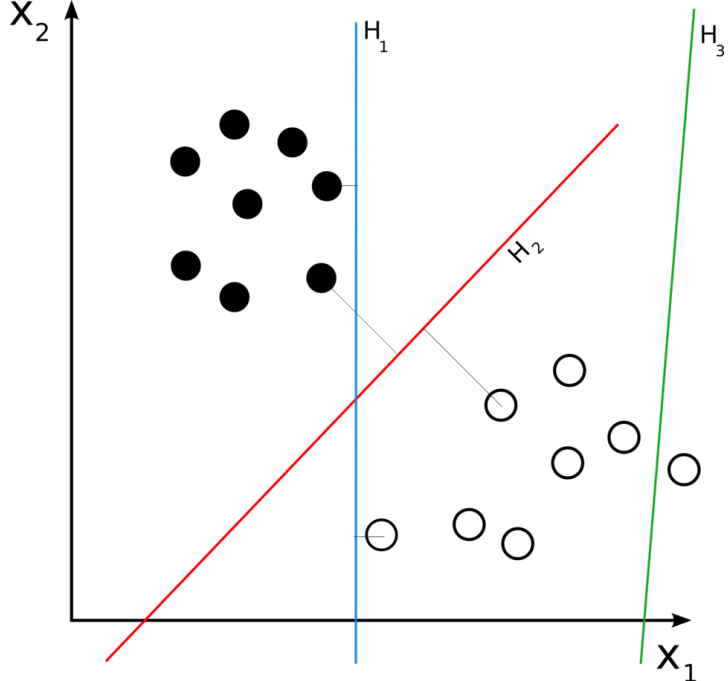
Metoda potpornih vektora (*Support Vector Machine*) je model nadgledanog učenja.

Ideja

Pronaći granicu odlučivanja sa najvećom marginom, odnosno onu koja pravi što veću udaljenost od najbližih uzoraka.

Za iste podatke može postojati više granica koje razdvajaju klase, a SVM bira onu koja ostavlja najveći razmak između klasa.

Zašto? Šta omogućava veća margina?



Potporni vektori

Potporni vektori su uzorci koji se nalaze najbliže granici odlučivanja.

- ▶ određuju položaj granice
- ▶ određuju širinu margine
- ▶ najviše utiču na odluku modela

Treniranje SVM-a kao **optimizacioni problem**: pronaći najbolju granicu odlučivanja.

SVM - algoritam

SVM traži granicu odlučivanja oblika:

$$w^T x + b = 0$$

Tokom treniranja model bira vrednosti parametara w i b tako da:

- ▶ klase budu što bolje razdvojene
- ▶ margina bude što šira
- ▶ greške budu kontrolisane

Ako su klase potpuno razdvojive, SVM može tražiti granicu bez grešaka.
Da li je u praksi tako?

Margine

- ▶ **Hard margin** - ne dozvoljava greške u klasifikaciji
- ▶ **Soft margin** - dozvoljava kontrolisane greške radi bolje generalizacije

Hiperparametar C kontroliše koliko model kažnjava greške.

- ▶ **manja kazna** → šira margina, više dozvoljenih grešaka
- ▶ **veća kazna** → uža margina, manje dozvoljenih grešaka

Do čega može dovesti premala, a do čega prevelika kazna?

SVM - klasifikacija novih uzoraka

Pretpostavka: model je odredio granicu odlučivanja

Postupak

Za novi uzorak proverava sa koje strane granice se nalazi.

$$w^T x + b > 0 \Rightarrow \text{klasa A}$$

$$w^T x + b < 0 \Rightarrow \text{klasa B}$$

Predikcija zavisi od položaja uzorka u odnosu na naučenu granicu odlučivanja.

Kernel

Kernel funkcija koja omogućava SVM-u da formira nelinearnu granicu odlučivanja.

Ideja

- ▶ podaci se posmatraju u drugačijem prostoru u kom ih je lakše razdvojiti
- ▶ nije potrebno ručno dodavati nove atribute

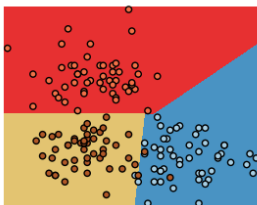
Kernel omogućava linearnom modelu da se ponaša kao nelinearan.

Najčešći kerneli:

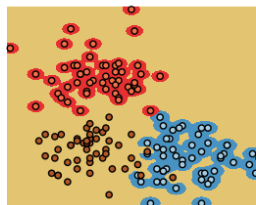
- ▶ **linear** - linearna granica odlučivanja
- ▶ **poly** - polinomijalna granica odlučivanja
- ▶ **rbf** - fleksibilna nelinearna granica odlučivanja

Uticaj kernela na granicu odlučivanja

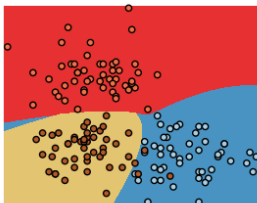
SVC with linear kernel



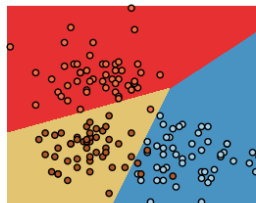
SVC with RBF kernel



SVC with polynomial (degree 3) kernel



LinearSVC (linear kernel)



SVM - zaključak

Najvažniji hiperparametri kod SVM-a su:

- ▶ **kernel** - oblik granice odlučivanja,
- ▶ **C** - odnos između širine margine i grešaka,
- ▶ **gamma** - fleksibilnost granice kod RBF kernela.

Prednosti

- ▶ koristan kod velikog broja atributa
- ▶ podržava nelinearne granice

Ograničenja

- ▶ sporost na velikim skupovima
- ▶ slabija interpretabilnost
- ▶ osetljivost na šum i preklapanja

SVM je dobar kada granica između klasa nije trivijalna i skup podataka nije prevelik.

SVM nije model koji se bira automatski za svaki problem!

Stablo odlučivanja

Stablo odlučivanja (*Decision Tree*) je model **nadgledanog učenja** koji predikciju formira kroz niz jednostavnih pitanja.

- ▶ Da li atribut zadovoljava određeni uslov?
- ▶ Koji uslov najbolje razdvaja podatke?
- ▶ Kada je odluka dovoljno sigurna?

Ideja

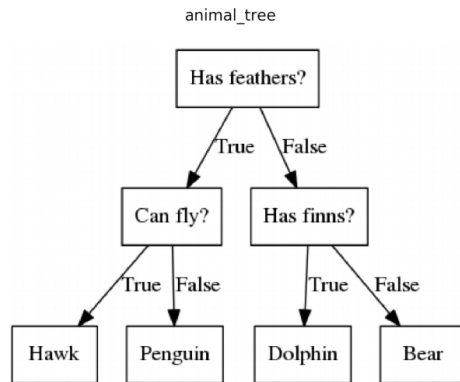
Stablo odlučivanja deli podatke na manje podskupove, pri čemu se svaka podela zasniva na jednom atributu.

Pravilo odlučivanja predstavlja put od korena do lista stabla.

Stablo odlučivanja - struktura

Osnovni elementi stabla su:

- ▶ **koren** - početni čvor
- ▶ **unutrašnji čvor** - pitanje nad atributom
- ▶ **grana** - ishod pitanja
- ▶ **list** - konačna predikcija



Stablo odlučivanja - predikcija

Za svaki novi uzorak stablo proverava uslove redom.

Pravila odlučivanja - primer:

$$atribut\ 1 \leq vrednost \quad \wedge \quad atribut\ 2 > vrednost \quad \rightarrow \quad klasa$$

Svaki prolazak kroz stablo odgovara jednom pravilu oblika:

ako su određeni uslovi ispunjeni, onda je predikcija određena klasa.

Zbog ovakve strukture, stabla su često jednostavna za objašnjenje.

Stablo odlučivanja - algoritam

Stablo odlučivanja se gradi rekurzivnim deljenjem podataka.

Algoritam:

- ▶ svi trening podaci nalaze se u korenu stabla
- ▶ bira se atribut i uslov koji najbolje razdvajaju podatke
- ▶ podaci se dele na manje podskupove
- ▶ **postupak se ponavlja za svaki novi čvor**
- ▶ kriterijum zaustavljanja: dalja podela više nema smisla

Kada zaustaviti grananje?

Kako se bira najbolja podela?

Izbor najbolje podele

Cilj

Cilj podele je da se smanji pomešanost klasa u novodobijenim čvorovima, odnosno, da oni budu što **čistiji**.

Čist čvor = većina uzoraka pripada istoj klasi $\rightarrow$ sigurnija odluka

Za merenje čistoće pri izboru podele uglavnom se koriste:

- ▶ **Gini nečistoća** - mera pomešanosti klasa u čvoru
- ▶ **entropija** - mera neuređenosti ili nesigurnosti u čvoru

Šta ako su listovi potpuno čisti?

Overfitting

Hiperparametri

`max_depth, min_samples_split, min_samples_leaf, max_features`

Stablo odlučivanja može previše da se prilagodi trening podacima ako:

- ▶ formira previše specifična pravila
- ▶ deli podatke sve do veoma malih grupa
- ▶ počne da uči šum umesto opštih obrazaca

Kako se overfitting manifestuje u rezultatima modela?

Kako se overfitting može sprečiti ili barem ublažiti?

Stablo odlučivanja - zaključak

Prednosti

- ▶ laka interpretacija
- ▶ prirodno hvata nelinearne odnose
- ▶ ne zahteva skaliranje atributa
- ▶ ukazuje na važnost atributa

Ograničenja

- ▶ sklonost overfitting-u
- ▶ nestabilnost modela
- ▶ visoka varijansa
- ▶ često slabije performanse kao samostalan model

Stablo odlučivanja je razumljiv i brz model, ali jedno stablo obično nije dovoljno stabilno za pouzdanu predikciju.

Kako se može ublažiti nestabilnost jednog stabla odlučivanja?
Može li jedno stablo biti polazna osnova za složeniji model?

Ansambl metode

Ansambl metode kombinuju predikcije više pojedinačnih modela radi dobijanja stabilnije i pouzdanije konačne odluke.

Pojedinačni modeli koji čine ansambl nazivaju se **bazni modeli**.

Osnovna ideja:

- ▶ jedan model može imati izražene slabosti
- ▶ više modela može smanjiti uticaj pojedinačnih grešaka
- ▶ konačna predikcija se dobija kombinovanjem više predikcija

Način kombinovanja predikcija zavisi od tipa problema.

Kako kombinovati predikcije kod klasifikacije, a kako kod regresije?

Bagging i boosting

Bagging

- ▶ modeli se treniraju nezavisno nad različitim podskupovima podataka
- ▶ cilj: smanjenje varijanse
- ▶ primer: Random Forest

Boosting

- ▶ modeli se treniraju sekvencijalno
- ▶ svaki sledeći model popravlja greške prethodnih
- ▶ cilj: postepeno smanjenje greške
- ▶ primer: Gradient Boosting

Oba pristupa uvode dodatne **hiperparametre** od kojih će kvalitet rezultata značajno zavisiti.

Random Forest

Random Forest je ansambl metoda zasnovana na većem broju stabala odlučivanja.

Ideja

Model formira različita stabla i kombinuje njihove predikcije uz uvođenje slučajnosti.

Osnovni postupak:

- ▶ formira se veći broj različitih trening podskupova
- ▶ nad svakim podskupom trenira se posebno stablo
- ▶ pri podeli čvorova razmatra se samo deo atributa
- ▶ svako stablo daje sopstvenu predikciju
- ▶ konačna predikcija se dobija kombinovanjem predikcija stabala

Slučajnost u izboru podataka i atributa čini stabla međusobno različitim.

Random Forest - karakteristike

Klasifikacija	Regresija
▶ svako stablo glasa za jednu klasu ▶ konačna predikcija: klasa sa najviše glasova	▶ svako stablo daje brojčanu vrednost ▶ konačna predikcija: prosečna vrednost

Random Forest smanjuje varijansu modela jer:

- ▶ koristi više različitih stabala
- ▶ svako stablo vidi drugačiji deo podataka
- ▶ svako stablo razmatra različite atribute pri podelama
- ▶ greška jednog stabla ima manji uticaj na konačnu odluku

Napomena: veća vremenska i memorijska zahtevnost

Gradient Boosting

Ideja

Gradient Boosting sekvencijalno gradi jak model postepenim dodavanjem jednostavnih modela koji u svakoj iteraciji ispravljaju greške iz prethodnih.

Osnovni postupak:

- ▶ postavlja se jednostavna početna predikcija
- ▶ računaju se greške trenutnog modela
- ▶ trenira se novo stablo odlučivanja koje pokušava da popravi te greške
- ▶ nova korekcija se dodaje postojećem modelu
- ▶ postupak se ponavlja kroz više iteracija

Za razliku od RF, **modeli u Gradient Boosting-u zavise jedni od drugih.**

Gradient Boosting - karakteristike

Gradient Boosting je **osetljiv na podešavanje hiperparametara**.

Hiperparametri

- ▶ `learning_rate` - koliko svako novo stablo utiče na konačnu predikciju
- ▶ `n_estimators` - koliko se stabala dodaje u ansambl

Kako manji, odnosno veći `learning_rate` utiče na performanse modela?

Kako broj stabala utiče na složenost modela?

Kako se ova dva hiperparametra mogu međusobno kompenzovati?

Optimizacija hiperparametara

Hiperparametri određuju ponašanje modela i zadaju pre treniranja.

Isti algoritam može dati veoma različite rezultate za različite vrednosti hiperparametara!

Primeri hiperparametara:

- ▶ SVM: `C`, `gamma` kod SVM modela
- ▶ stabla odlučivanja: `max_depth`, `min_samples_leaf`
- ▶ Random Forest model: `n_estimators`, `max_features`
- ▶ Gradient Boosting model: `learning_rate`, `n_estimators`

Šta su parametri, a šta hiperparametri kod linearnog modela?

Podešavanje hiperparametara

Cilj

Postići bolju ravnotežu između tačnosti, složenosti i generalizacije modela.

Do čega mogu dovesti neadekvatno podešene vrednosti hiperparametara?

Uobičajen postupak:

1. definiše se skup mogućih vrednosti
2. trenira se više verzija istog modela
3. svaka verzija se vrednuje na validacionim podacima
4. bira se kombinacija sa najboljim validacionim rezultatom

Podešavanje hiperparametara mora biti deo **validacione** procedure, a ne testiranja!

Grid Search

Grid Search sistematično isprobava sve kombinacije predefinisanih vrednosti hiperparametara.

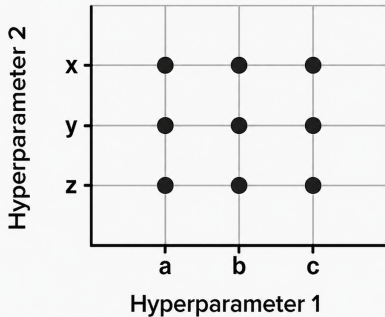
Prednost

- ▶ detaljna i kompletna pretraga
- ▶ pogodno za manji prostor pretrage

Ograničenje

- ▶ može biti računski skupo i sporo

Hyperparameter_One = [a, b, c]
Hyperparameter_Two = [x, y, z]
Hyperparameter_X = [i, j, k]



Random Search

Random Search nasumično bira kombinacije hiperparametara iz zadatog prostora pretrage.

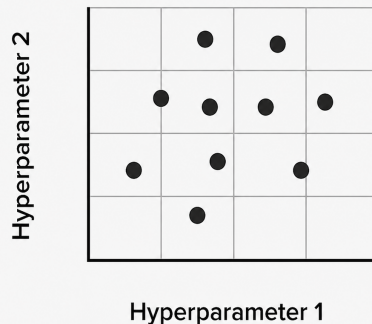
Prednost

- ▶ računarska efikasnost
- ▶ pogodno za veći prostor pretrage

Ograničenje

- ▶ ne proverava sve moguće kombinacije

Hyperparameter_One = random.num (range)
Hyperparameter_Two = random.num (range)
Hyperparameter_X = random.num (range)



Validacija

Hiperparametri se podešavaju na osnovu validacionih rezultata.

Uobičajen postupak:

- ▶ trening skup - obučavanje modela
- ▶ validacioni skup - izbor hiperparametara
- ▶ test skup - finalna procena modela

Test skup se koristi tek nakon izbora modela i hiperparametara!

Šta ako je test skup upotrebljen tokom podešavanja modela?

Poređenje modela

Poređenje modela je smisleno akko su **evaluירani pod istim uslovima**:

- ▶ ista podela podataka
- ▶ isti preprocessing postupak
- ▶ iste metrike
- ▶ ista validaciona procedura
- ▶ testiranje tek nakon razvoja modela

Rezultat modela je relevantan samo u kontekstu evaluacionog protokola po kome je dobijen.

Pri odabiru najboljeg modela, potrebno je uzeti u obzir metriku, tip problema, stabilnost rezultata, interpretabilnost i složenost modela.